Guía de instalación y despliegue
API de Asistencia - Plataforma InfoUNSA

Requisitos

Instalar previamente:

- Node.js (versión compatible con el proyecto)(la última LTS)

  https://nodejs.org/dist/v24.18.0/node-v24.18.0-x64.msi

- npm
- MySQL Server 8.0.46.0

  https://dev.mysql.com/get/Downloads/MySQLInstaller/mysql-installer-community-8.0.46.0.msi

- Git

  https://github.com/git-for-windows/git/releases/download/v2.55.0.windows.2/Git-2.55.0.2-64-bit.exe

1. Clonar el repositorio
git clone https://gitlab.com/Yodetcozco/plataforma-infounsa

```
git fetch origin
git switch --track origin/develop-localhost
```

2. Instalar dependencias
Desde la carpeta: ***"apps/asistencia/api"*** instalar dependencias con el comando "npm install"
3.configurar la base de datos

a. Abre MySQL Command Line Client o tu gestor de base de datos preferido en el servidor.
b. Ejecuta los siguientes comandos para crear las bases de datos vacías: sql

```sql
CREATE DATABASE infounsa_asistencia CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
CREATE DATABASE infounsa_inventario CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

c. Crea un usuario dedicado para las APIs (o configura uno existente):

```SQL
CREATE USER 'infounsa'@'localhost' IDENTIFIED BY
'infounsa_2026';
GRANT ALL PRIVILEGES ON infounsa_asistencia.* TO
'infounsa'@'localhost';
GRANT ALL PRIVILEGES ON infounsa_inventario.* TO
'infounsa'@'localhost';
FLUSH PRIVILEGES;
```

d. Crear todas las tablas de la base de datos
Esto se puede desde MySQL Command Line Client o del cmd usando las credenciales que se configuraron al instalar mysql (SIN COMILLAS)

```SQL
USE infounsa_asistencia;
SOURCE "c:/ruta del
repositorio/plataforma-infounsa/apps/asistencia/database/init/001_create_attend
ance_schema.sql";
```

4. Configurar el archivo .env

API Asistencia
Crea el archivo
apps/asistencia/api/.env:

```None
PORT=3000
DATABASE_URL="mysql://infounsa:infounsa_2026@localhost:3306/infou
nsa_asistencia"
ASISTENCIA_ALLOWED_ORIGINS="http://localhost:3000,http://127.0.0.
1:3000" # Configura otros secrets requeridos por el backend de
Asistencia
```

esta parte puede dar algunos problemas de codificación
así que desde la carpeta api de nuestro proyecto (\apps\asistencia\api) ejecutamos el siguiente comando desde powershell(de preferencia como administrador por los permisos), obvio cambiando puerto y demás variables (usuario, contraseña, base de datos y el puerto)

```
None
@"
PORT=3000
DATABASE_URL=mysql://infounsa:infounsa_2026@localhost:3306/infounsa_asistencia
ASISTENCIA_ALLOWED_ORIGINS=http://localhost:3000,http://127.0.0.1:3000
"@ | Set-Content .env -Encoding utf8
```

Luego para verificar que todo esté correcto ejecutamos el siguiente comando

```
None
node -e "require('dotenv').config(); console.log(process.env.DATABASE_URL)"
```

y nos debe mostrar las variables que se usaron para la conexión

5. Verificar la conexión

Ejecutar en la carpeta “apps/asistencia/api”

```
None
npx prisma db pull
```

Debe aparecer algo similar a:

Introspected XX models

Si aparece:

Can't reach database o Access denied
posibles problemas con el .env posiblemente un error con la contraseña del usuario infounsa que se creó en el paso 3.c

6. Generar el cliente Prisma

Cada vez que se haga algún cambio en schema.prisma se ejecuta este comando desde ***“apps/asistencia/api”***

```
None
npx prisma generate
```

7. Compilación limpia (recomendado)

Antes de compilar por primera vez o después de cambios importantes(estos comandos se ejecutan dentro de ***apps/asistencia/api***):

```
None
Remove-Item dist -Recurse -Force
Remove-Item tsconfig.tsbuildinfo -Force
```

estos dos comandos son equivalentes a borrar la carpeta dist(***apps/asistencia/api/dist***) y el archivo tsconfig.tsbuildinfo (***apps/asistencia/api/***tsconfig.tsbuildinfo)

Luego:

```
None
npx tsc
```

para compilar y generar todo limpiamente

Esto evita inconsistencias entre archivos antiguos y nuevos. esta parte es importante porque sino pueden haber varios archivos que posiblemente no estén compilados y van a saltar errores de archivos inexistentes en la compilación, en simples palabras, los archivos de la carpeta /src (código fuente) a dist(archivos compilados o ejecutables)

Este paso se debe repetir cada vez que se hagan cambios al código fuente(src), ya que se limpia la compilación anterior y se vuelve a compilar en ejecutables (dist) con los cuales se levanta el api

8. Levantar la API

Ejecutar:

```
None
node dist/main.js
```

Debe aparecer:
Nest application successfully started
Asistencia API escuchando en puerto 3000

9. En caso de posibles problemas con el cliente prisma seria mejor estandarizar esto del **schema.prisma**

```
None
generator client {
  provider     = "prisma-client"
  output       = "../src/generated/prisma"
  moduleFormat = "cjs"
}
```

en simples palabras cambiarlo por

```
None
generator client {
  provider     = "prisma-client-js"
}
```

También cambiar los import de prisma client en los archivos

apps/asistencia/api/prisma/seed.ts
apps/asistencia/api/src/prisma/prisma-service.ts

de

```
None
import { PrismaClient } from '../generated/prisma/client';
```

a

```
None
import { PrismaClient } from '@prisma/client';
```

10. Regla de Firewall
Esta parte es importante para que los dispositivos de la red no tengan problemas al conectarse

Con la siguiente línea de comandos creamos la regla desde el cmd o powershell (como administrador de preferencia para evitar problemas con los permisos) cambiando el nombre si se cree conveniente y más importante aún el puerto

```
None
New-NetFirewallRule ` -DisplayName "Asistencia API 3000" `
-Direction Inbound ` -Protocol TCP ` -LocalPort 3000 ` -Action
Allow
```

Para verificar su existencia usamos lo siguiente

```
None
Get-NetFirewallRule | Where-Object DisplayName -like "*3000*"
```

11. Convertirlo en un servicio

a. **Descargar NSSM**

Descarga la versión estable:

https://nssm.cc/download

Extrae el ZIP, por ejemplo en:

```
None
C:\Herramientas\nssm\
```

Dentro tendrás:

```
None
C:\Herramientas\nssm\win64\nssm.exe
```

b. **Instalar el servicio**

Ve a la carpeta donde está `nssm.exe`:

```
None
cd C:\Herramientas\nssm\win64
```

Ejecuta:

```
None
.\nssm.exe install AsistenciaAPI
```

Se abrirá una ventana.

c. Configurar el servicio

Path

Debe apuntar a Node:

Normalmente:

```
None
C:\Program Files\nodejs\node.exe
```

Startup directory

Debe ser la carpeta de la API:

```
None
C:\Plataforma\plataforma-infounsa\apps\asistencia\api
```

Arguments

```
None
dist\main.js
```

Quedará algo así:

```
None
Path:
C:\Program Files\nodejs\node.exe

Startup directory:
C:\Plataforma\plataforma-infounsa\apps\asistencia\api

Arguments:
dist\main.js
```

Luego presionar:

```
None
Install Service
```

d. Iniciar el servicio

Puedes hacerlo desde PowerShell:

```
None
Start-Service AsistenciaAPI
```

o

```
None
sc start AsistenciaAPI
```

e. Verificar

```
None
Get-Service AsistenciaAPI
```

Debe salir:

```
None
Status : Running
```

f. Revisar el Administrador de Servicios

Ejecuta:

```
None
services.msc
```

Deberías ver:

```
None
AsistenciaAPI
```

Con estado:

```
None
Running
```

g. Configurar inicio automático

Puedes hacerlo desde `services.msc` o con:

```
None
Set-Service AsistenciaAPI -StartupType Automatic
```

Así, cuando reinicies el Windows Server, la API arrancará sola.

h. Detener o reiniciar

Detener:

```
None
Stop-Service AsistenciaAPI
```

Reiniciar:

```
None
Restart-Service AsistenciaAPI
```